Convoy: shipment tracking prototype

Convoy gives a manufacturer's customers **one place to see where an order is and when it arrives**, and gives its ops team **a short list of what to chase today**. It is a clickable prototype on mocked data. The AI is simulated with rules.

The cut I chose and why

I built two roles, and **every screen answers one question**. The customer asks "is my order on time, and when does it arrive?" They get a shipment list, a detail page and a notification feed. Ops asks "which shipments do I chase first?" They get Today, an exceptions queue, a portfolio table and a detail page that shows the raw operator feed next to the milestones.

Ops covers two of the four sites, by origin or destination. **One shipment sits outside that perimeter on purpose**: its URL says "Shipment not found" and search doesn't return it. Nordvik, the main demo customer, receives goods in Norway, outside the EU customs union. That gives the demo **real import customs at Oslo** and a Norwegian broker posting statuses in Norwegian. Sea arrivals inside the EU only go through "Port formalities (Union status check)" with a T2L proof of Union status, because a customs declaration only exists outside the union. I wanted the customs flow to match how it really works.

Left out, and why:

- A map. A good one costs hours and a cheap one looks generic. The timeline carries the same information.
- Real operator integrations, document upload, analytics, dark mode and a translated UI. The brief is frontend only, and none of these changes the answer to the two questions.
- Automated tests. I spent the hours on the product and checked by hand: lint, build, both roles at 1440 and 360 px, keyboard navigation and reduced motion. This is a real gap.

Where and how the AI adds value

There are seven features, each placed where someone needs it.

Feature	Where	Based on
Exception management	Ops Today, exceptions queue	Delays, customs holds, silent feeds, missing documents and port incidents. Each row proposes one action.
Arrival estimates	Shipment detail, both roles	Confirmed delays plus typical customs time, shown with confidence and reason.
Consolidation	Timeline, ops detail	Seven operator vocabularies ("Gate-in", "VSL DEP", "Fortollet") mapped to one milestone set, with the raw feed alongside.
Daily summary	Ops Today	Live counts and ranked next actions, recomputed after every action.
Natural-language search	Ops portfolio	"shipments to France this week running late" becomes three filter chips. Words it doesn't understand are marked as ignored, and one control clears everything.

Feature	Where	Based on
Proactive notices	Exceptions queue, customer banner, feed	Order, route, estimate, confidence and cause. Ops edits before sending.
Drafting in the recipient's language	Message dialogs	The customer's account language or the operator's language (French for Rhone), with the reason shown.

Outputs are branded Convoy, not "AI". I want people to **trust an answer because they can see the facts behind it**. Every output carries a stamp saying what it was built from and a "Why" line that names those facts. The interface never says "prototype" or "simulated". The README and this document do.

The rule I cared about most is that **a confirmed date and a Convoy date never look the same**. Operator events are solid, planned legs are dashed, and the span an estimate moved sits on an amber band. A single `Estimate` component renders every arrival date. If nothing moved the date, it says "Planned", not "Estimated". An unknown operator status shows as "Not mapped", and a quiet feed tells the customer "Awaiting update". I'd rather show a gap than guess.

The live feed covers freshness. Seven scripted operator events arrive one every four seconds on the demo clock, with pause, resume and reset. Each lands with a highlight that fades, a toast linking to the order, and a new "Last updated" time in the header. When Nordhaul posts its own ETA for order 12466, Convoy **treats it as an operator fact**: solid, labelled "Operator ETA", and the earlier Convoy estimate moves into "ETA history".

Every message dialog has a "To" line with recipient and channel: email for operators, Convoy inbox plus email for customers. **Documents follow ownership.** The manufacturer's own (invoice, packing list, T2L) are sent with the ERP file attached, through a fake picker. The carrier's bill of lading or CMR and the broker's customs declaration are requested from them. **Customers never get a document request.** They download existing documents as one-page PDFs built in the browser. Customer notices are capped at 600 characters and clamped to three lines.

The language switch **only goes towards English**, the shared language of the trade. "Write in English instead" regenerates the draft and "Back to French" undoes it. An English draft offers no switch, because with many possible languages Convoy couldn't know which one to offer.

Technical approach

The store holds facts, and everything else is derived from them. The stack is React 19, Vite, react-router for real URLs and plain CSS with custom-property tokens. There is no backend, UI kit or state library: a prototype this size doesn't need them, and real URLs let you open any shipment straight from a link.

The store (a context plus a reducer) holds operator events, the plan, documents and what ops did. Risk, estimates, summary, search and drafts are pure functions in `src/ai/`, read through selectors. So when ops acts on a shipment, the customer view updates with no extra wiring.

Risk weights are plain numbers: +4 for a customs hold, +3 for 24 hours of silence, +2 per day of slip, capped at +6. **The same inputs always give the same ranking.** The clock is fixed at 13 Oct 2026, 14:30, and the live feed stamps its events against it. The data (24 shipments, seven operators, four sites) is identical on any machine.

In production, an LLM would map free-text statuses to milestones against a fixed schema. The current rules stay as a fallback, and anything with low confidence lands in "Not mapped". Estimates would come from a model trained on lane history and port feeds, with thresholds deciding whether to show a date, a range or "no update yet". A model drafts notices and **a person approves each one**. Search returns the same filter object the dropdowns use, so the chips stay inspectable. A labelled set of real statuses and past delays scores every change before release. Events arrive by webhook, with polling as a fallback.

Assumptions, limitations and next steps

Assumptions: one customer and one ops persona are enough to show scoping. The UI is in English, with metric units and EUR. Operators post timestamped events. The demo runs locally, so there is no deployment or auth.

Limitations:

- Reloading resets all data.
- The live feed is a seven-event script.
- Search understands a fixed set of phrases.
- Risk and estimates are rules, not models.
- Drafts exist in English and French only.
- No auth and no automated tests.

Next steps, in order:

1. Real events behind a thin API with webhooks.
2. An evaluation set for the status mapper, before any LLM touches customer-facing text.
3. Tests for the risk and search rules, which are the cheapest to cover.
4. Notification preferences, document upload and a map, once I know what customers ask for.

I used AI coding tools (Claude Code) throughout. I made the scope, product and visual decisions and reviewed what they produced.